

Diagonal Difference:-

Code:-

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.toList;
class Result {
    public static int diagonalDifference(List<List<Integer>> arr) {
        int n = arr.size();
        int diagonal1 = 0;
        int diagonal2 = 0;
        for (int i = 0; i < n; i++) {
            diagonal1 += arr.get(i).get(i);
            diagonal2 += arr.get(i).get(n - 1 - i);
        }
        return Math.abs(diagonal1 - diagonal2);
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(
            new InputStreamReader(System.in)
        );
        BufferedWriter bufferedWriter = new BufferedWriter(
            new FileWriter(System.getenv("OUTPUT_PATH"))
        );
        int n = Integer.parseInt(bufferedReader.readLine().trim());
        List<List<Integer>> arr = new ArrayList<>();
        IntStream.range(0, n).forEach(i -> {
            try {
                arr.add(
                    Stream.of(
                        bufferedReader.readLine()
                            .replaceAll("\\s+$", "")
                            .split(" ")
                    )
                    .map(Integer::parseInt)
                    .collect(toList())
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
        int result = Result.diagonalDifference(arr);
        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

Output:-

✓ **Test case 0**

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

1	3
2	11 2 4
3	4 5 6
4	10 8 -12

[Download](#)

Expected Output

1	15
---	----

[Download](#)

Transpose Matrix:-

Code:-

```
class Solution {
    public int[][] transpose(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        int[][] result = new int[n][m];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                result[j][i] = matrix[i][j];
            }
        }
        return result;
    }
}
```

Output:-

